

An Empirical Evaluation of Code Review Techniques Using a Weighted Code Quality Score

1st Priyanka Kadam

*Dept. Computer Engineering
Vishwakarma Institute of
Technology*

Pune-411037, India

Priyanka.kadam1@vit.edu

2nd Anushka Auti

*Dept. Computer Engineering
Vishwakarma Institute of
Technology*

Pune-411037, India

anushka.auti24@vit.edu

3rd Tejas Bankar

*Dept. Computer Engineering
Vishwakarma Institute of
Technology*

Pune-411037, India

tejas.bankar24@vit.edu

4th Yash Bhandari

*Dept. Computer Engineering
Vishwakarma Institute of
Technology*

Pune-411037, India

Yash.bhandari24@vit.edu

5th Lakshit Barekar

*Dept. Computer Engineering
Vishwakarma Institute of
Technology*

Pune-411037, India

lakshit.barekar24@vit.edu

Abstract—Code review is a widely used practice in software development for identifying defects and improving overall code quality. Different review approaches such as peer review, pull-based review, and tool-assisted review are commonly applied in modern development workflows. However, objectively measuring the effectiveness of these review techniques remains a challenge. This research presents a Code Review Analytics System designed to evaluate different code review techniques using a quantitative metric called the Weighted Code Quality Score (WQS). The proposed system performs static code analysis on Python source code using the Pylint tool and categorizes detected issues into three severity levels: critical, major, and minor defects. Based on these defect classifications, the system computes the WQS to represent the quality impact of the detected issues. The system also stores historical review results and generates visualizations that help analyze defect distribution and compare review techniques. Experimental results demonstrate how different review methods detect varying numbers and types of defects. The proposed framework provides a structured approach for analyzing code review effectiveness and supporting data-driven software quality improvement.
*Index Terms—*Migraine Relief, Headset, Vibration Therapy, Sound Therapy, Breathing Exercises, Non-invasive, Smart Wearable Device, Arduino Based control, Real-Time monitoring, Web based control, Stress Reduction, Drugfree Treatment, Personalized Therapy.

I. INTRODUCTION

Ensuring high software quality is one of the most important goals in modern software engineering. As software systems become more complex, detecting defects early during development is essential to avoid costly failures in later stages. Code review has emerged as one of the most effective techniques for identifying bugs, improving readability, and ensuring adherence to coding standards.

Code review involves examining source code to identify potential problems and suggest improvements. Several approaches to code review exist, including peer review, pull-based review, and tool-assisted review using automated static analysis tools. Peer review typically involves developers manually examining each other's code, while pull-based review is commonly performed through version control systems such as GitHub. Tool-assisted review uses automated software tools to analyze code and detect issues. Although these techniques are widely used, comparing their effectiveness in a structured and measurable way remains difficult. Different review methods may detect different types of defects, and there is often no consistent metric to evaluate their impact on code quality. To address this challenge, this study proposes a Code Review Analytics System that evaluates code review techniques using a quantitative metric known as the Weighted Code Quality Score (WQS). The system analyzes Python source code using static analysis tools, classifies defects according to severity levels, and calculates a score representing overall code quality. The results are stored and visualized to allow comparison between different review approaches. The main objective of this research is to provide a systematic framework for analyzing and comparing code review practices in an academic environment.

II. LITERATURE REVIEW

Code review is an essential practice in software engineering aimed at improving code quality, detecting defects, and ensuring adherence to coding standards. Over the years, several studies have explored different code review techniques and their effectiveness in identifying software defects.

Rigby and Storey [1] examined peer review practices in open-source software projects and highlighted how collaborative review processes help in early defect detection and knowledge sharing among developers. Their study emphasized the importance of human involvement in identifying logical errors and improving code maintainability. However, their work also pointed out that manual reviews can be inconsistent and dependent on reviewer expertise.

Bacchelli and Bird [2] analyzed modern code review processes in industrial environments and discussed the challenges associated with pull-based review systems. Their research demonstrated that while pull-based reviews facilitate collaboration through version control systems, they often suffer from delays, communication gaps, and varying review quality. This highlights the need for systematic approaches to evaluate review effectiveness.

Static code analysis tools have also been widely used to support automated code reviews. According to Ayewah et al. [7], static analysis tools can efficiently detect a wide range of coding errors, including syntax issues, potential bugs, and violations of coding standards. These tools provide consistent and fast feedback, making them valuable in large-scale software development. However, they are limited in detecting logical and runtime errors, which often require human judgment.

McCabe [4] introduced the concept of software complexity measurement, which provides insight into the maintainability and reliability of code. Complexity metrics help in identifying sections of code that are more prone to defects. Similarly, McConnell [5] and Sommerville [6] emphasized the importance of structured coding practices and quality assurance techniques in improving software reliability.

Despite these contributions, most existing studies focus either on individual review techniques or on defect detection without providing a unified framework for comparison. There is a lack of standardized metrics that can quantitatively evaluate the effectiveness of different code review methods. Simply counting the number of defects does not accurately reflect their impact on software quality.

To address this gap, the present study proposes a Code Review Analytics System that integrates static code analysis with a quantitative metric called the Weighted Code Quality Score

(WQS). Unlike traditional approaches, the proposed system categorizes defects based on severity and assigns weights to reflect their impact. This enables a more structured and objective comparison of different code review techniques, contributing to data-driven software quality evaluation.

III. PROBLEM STATEMENT

Code review is an important practice used in software development to detect defects, improve code readability, and maintain coding standards. Developers commonly use different review techniques such as peer review, pull-based review through version control systems, and automated tool-assisted reviews using static analysis tools. Each of these techniques contributes to software quality improvement in different ways. However, evaluating the effectiveness of these review techniques remains a challenge. In many development environments, code review results are based on subjective opinions or isolated analysis, which makes it difficult to objectively compare different review methods. Additionally, most existing tools focus only on detecting issues in code without providing a structured framework to analyze and compare the impact of different review approaches on overall code quality. Another challenge is the lack of a standardized metric that can measure code quality based on the severity of detected defects. Simply counting the number of defects does not accurately represent the impact of those defects on software reliability and maintainability. Therefore, there is a need for a system that can systematically analyze code review results, categorize defects based on severity, and provide a quantitative method for evaluating and comparing different code review techniques. This research addresses this problem by developing a Code Review Analytics System that uses static code analysis to detect defects and calculates a Weighted Code Quality Score (WQS) to measure the quality impact of those defects.

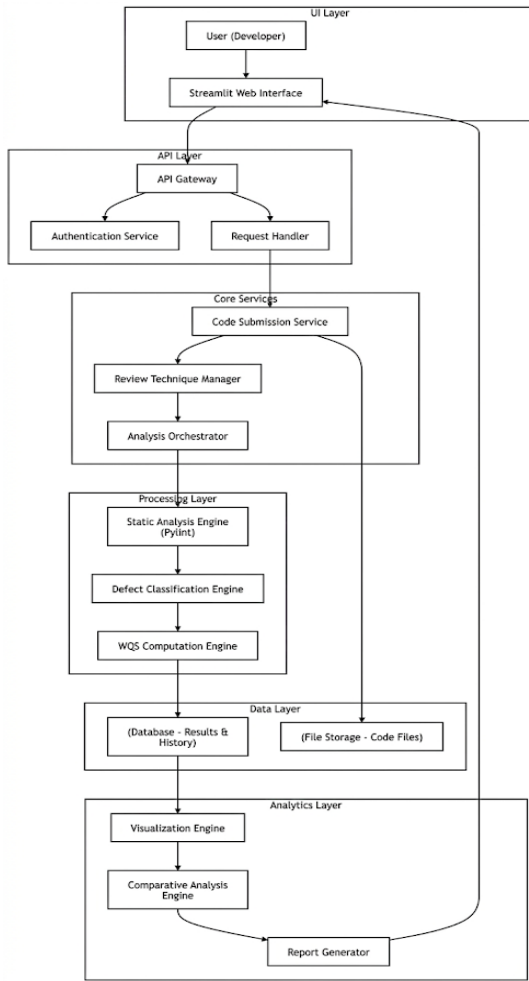


Fig1 System architecture diagram

IV. OBJECTIVES

The main objective of this research is to study and evaluate different code review techniques and their impact on software quality. Code review plays a significant role in identifying defects during the software development process and helps improve maintainability, reliability, and readability of source code. However, comparing the effectiveness of different review methods is difficult without a structured evaluation framework. Therefore, this study aims to develop a system that can analyze code review outcomes and measure the quality of code using a quantitative metric. The proposed system uses static code analysis to detect potential issues in Python source code and categorizes the detected defects according to their severity levels. Based on this classification, a Weighted Code Quality Score (WQS) is calculated to represent the overall quality of the code. The system also records historical review results and provides visualization features that allow comparison between different code review techniques.

To design and develop a Code Review Analytics System for evaluating software code quality.

To detect potential defects in source code using static code analysis tools such as Pylint.

To classify detected issues into critical, major, and minor defect categories based on their severity.

To compute the Weighted Code Quality Score (WQS) for measuring the impact of detected defects on overall code quality.

To analyze and compare different code review techniques including peer review, pull-based review, and tool-assisted review.

To store review results and maintain historical records for further analysis and comparison.

To provide visualization of defect distribution and review outcomes for better understanding of software quality trends

V. METHODOLOGY

This research proposes a structured approach to evaluate different code review techniques using automated analysis and a quantitative scoring metric. The methodology consists of several stages including code submission, static code analysis, defect classification, score calculation, and result visualization. Each stage plays an important role in identifying defects and evaluating the effectiveness of code review practices. The system is designed to analyze Python source code files using a static analysis tool and categorize the detected issues based on severity. The identified defects are then used to compute a Weighted Code Quality Score (WQS), which represents the overall quality of the analyzed code. The results are stored and visualized to allow comparison between different review techniques.

5.1 Code submission module

The first stage of the methodology involves submitting source code for analysis. Users upload Python source code files through the system interface. During this stage, users also select the type of review technique to be evaluated, such as peer review, pull-based review, or tool-assisted review. The uploaded code serves as the input for further analysis. This module ensures that the system can handle multiple code files and record the review technique associated with each analysis.

5.2 Static Code Analysis

After the code file is uploaded, the system performs static code analysis using the Pylint tool. Static analysis examines the source code without executing it and identifies potential issues such as syntax errors, unused variables, coding standard

violations, and other programming mistakes. The analysis results are returned in structured form and used to detect different types of defects present in the code. This stage is essential for automatically identifying issues that may affect software quality.

5.3 Defect Classification

Once the analysis results are generated, the detected issues are categorized based on their severity level. The system divides defects into three categories: critical defects, major defects, and minor defects. Critical defects represent serious issues such as syntax errors or undefined variables that may cause the program to fail. Major defects represent warnings that may impact code maintainability or functionality. Minor defects correspond to coding style violations and convention issues that affect readability but do not significantly impact functionality. This classification helps in understanding the severity and impact of detected problems.

5.4 Weighted Code Quality Score Calculation

To evaluate the overall impact of detected defects, the system calculates a metric called the Weighted Code Quality Score (WQS). Each defect category is assigned a weight based on its severity. Critical defects receive the highest weight, followed by major defects and minor defects. The WQS is calculated using the following formula:

$$\text{WQS} = (\text{Critical} \times 5) + (\text{Major} \times 3) + (\text{Minor} \times 1)$$

This formula ensures that more severe defects have a greater influence on the final score. The resulting value represents the overall quality impact of the detected issues and allows different code review results to be compared effectively.

5.5 Data Storage and History Tracking

The system stores the analysis results in a structured dataset that includes the file name, selected review technique, number of detected defects in each category, and the calculated WQS. Maintaining historical records allows users to review past analyses and observe patterns in defect detection. This stored data also enables comparison of different review techniques across multiple code samples.

5.6 Visualization and Analysis

To make the analysis results easier to interpret, the system provides graphical visualization of defect distribution. Charts such as bar graphs and pie charts are generated to represent the number and severity of defects detected in each code file. These

visualizations help users quickly understand the defect patterns and evaluate the effectiveness of different review techniques. Visualization also supports comparative analysis of code quality results obtained through various review approaches.

5.7 Output Generation and Result Presentation

After the analysis and score calculation processes are completed, the system generates the final output for the user. The output includes the number of detected defects categorized into critical, major, and minor severity levels, along with the calculated Weighted Code Quality Score (WQS). These results provide a clear representation of the quality of the analyzed source code. The system presents the results through a graphical user interface developed using Streamlit. The interface displays a defect summary table showing the number of detected issues in each severity category. In addition to the numerical results, the system generates visual representations such as bar charts and pie charts to illustrate the distribution of detected defects. These visualizations help users quickly understand the severity and proportion of different types of defects present in the code. Furthermore, the system records the output results in a historical dataset. This dataset stores information such as the analyzed file name, the selected review technique, defect counts, and the calculated WQS. Maintaining this historical record allows users to review past analyses and compare the outcomes of different code review techniques over time. The generated output therefore provides both quantitative and visual insights into the quality of the analyzed code, supporting effective evaluation of code review practices and enabling easier interpretation of the analysis results.

Code Quality Analysis System

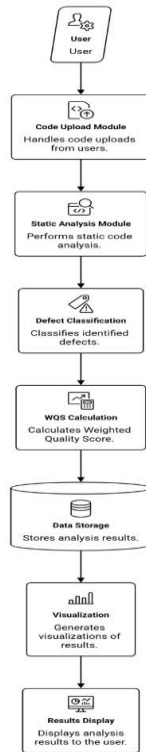


Fig.2 Workflow of code review technique

The architecture diagram presents the overall workflow of the proposed system. It illustrates how the uploaded code is processed through static analysis, defect classification, and WQS calculation modules, followed by storage and visualization of the analysis results.

VII. RESULTS AND DISCUSSION

The system was tested using several Python code files containing different types of defects. Each file was analyzed using the proposed framework and the results were recorded in the system's history database. An example of the recorded results is shown below.

File	Technique	Crit	Maj	Min	WQS
syntax.py	Tool-assisted	1	0	0	5
major_minor.py	Peer Review	0	0	4	4
undefined_variable.py	Pull-based	1	1	3	11

The results demonstrate that different review techniques may detect different types of defects. Tool-assisted review is particularly effective in identifying syntax errors and coding standard violations, while manual review techniques may detect logical issues that automated tools may miss.

The Weighted Code Quality Score provides a convenient way to compare review results and evaluate the severity of detected defects.

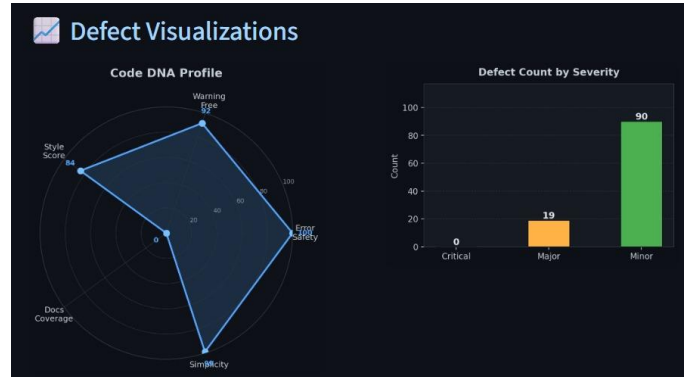


Fig 3. Defect Visualization

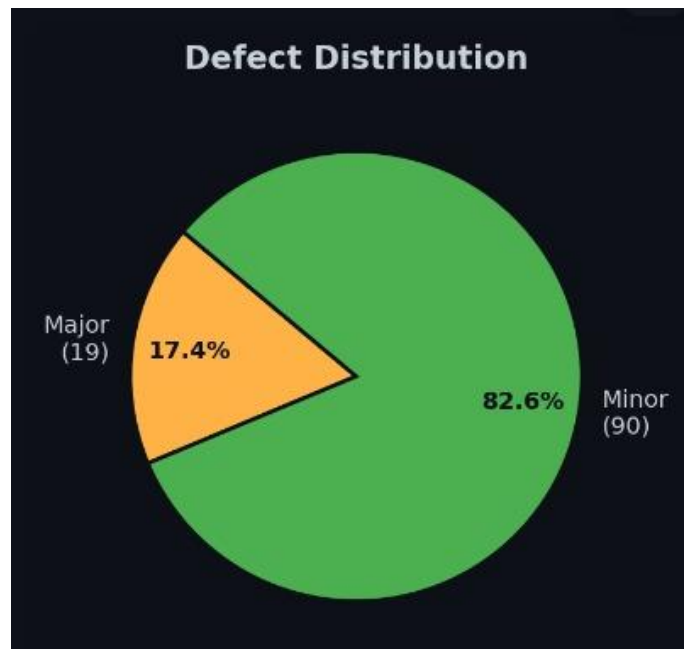


Fig 4. Defect Distribution

VIII. FUTURE SCOPE

The proposed Code Review Analytics System provides a structured approach for evaluating code quality and analyzing different code review techniques. However, there are several opportunities for further enhancement and expansion of the system in future work. One potential improvement is the integration of the system with version control platforms such as GitHub or GitLab. By connecting the system directly with pull request workflows, the tool could automatically analyze code changes submitted by developers and provide real-time feedback during the development process. This integration would make the system more practical for real-world software development environments.

Another possible enhancement is extending the system to support multiple programming languages. Currently, the system focuses on Python source code analysis using the Pylint tool. In the future, additional static analysis tools could be integrated to support other widely used languages such as Java, JavaScript, or C++. This would allow the system to be used in a broader range of software projects and development environments.

Furthermore, advanced techniques such as machine learning could be incorporated to improve defect prediction and review analysis. By analyzing historical review data, machine learning models could help predict defect severity, identify patterns in code quality issues, and recommend improvements to developers. Such intelligent features could enhance the effectiveness of code reviews and assist teams in prioritizing critical issues more efficiently.

Overall, these improvements could transform the proposed system into a more comprehensive and intelligent code review analytics platform capable of supporting modern software development practices.

IX. CONCLUSION

This research presented a Code Review Analytics System designed to evaluate the effectiveness of different code review techniques using a quantitative metric called the Weighted Code Quality Score. The system integrates static code analysis with defect classification and visualization capabilities to support structured analysis of software quality. The proposed framework enables users to analyze source code, categorize detected defects based on severity, and compute a score representing the overall quality impact of those defects. The system also maintains historical records of review results, allowing comparative analysis between different review approaches. Experimental testing shows that the system can effectively identify various types of defects and provide meaningful insights into code quality. The approach offers a practical method for evaluating code review practices in academic and experimental settings.

REFERENCES

- [1] C. Rigby and M. Storey, "Understanding Broadcast Based Peer Review on Open Source Software Projects," in *Proceedings of the 33rd International Conference on Software Engineering (ICSE)*, 2011, pp. 541–550.
- [2] A. Bacchelli and C. Bird, "Expectations, Outcomes, and Challenges of Modern Code Review," in *Proceedings of the 35th International Conference on Software Engineering (ICSE)*, 2013, pp. 712–721.
- [3] Pylint Documentation, "Pylint – Python Static Code Analysis Tool," Available: <https://pylint.org>
- [4] T. McCabe, "A Complexity Measure," *IEEE Transactions on Software Engineering*, vol. SE-2, no. 4, pp. 308–320, 1976.
- [5] S. McConnell, *Code Complete: A Practical Handbook of Software Construction*, 2nd ed., Microsoft Press, 2004.
- [6] I. Sommerville, *Software Engineering*, 10th ed., Pearson Education, 2016.
- [7] N. Ayewah, D. Hovemeyer, J. Morgenthaler, J. Penix, and W. Pugh, "Using Static Analysis to Find Bugs," *IEEE Software*, vol. 25, no. 5, pp. 22–29, 2008.
- [8] C. Rigby and M. Storey, "Understanding Broadcast Based Peer Review on Open Source Software Projects," in *Proc. 33rd International Conference on Software Engineering (ICSE)*, 2011, pp. 541–550.
- [9] A. Bacchelli and C. Bird, "Expectations, Outcomes, and Challenges of Modern Code Review," in *Proc. 35th International Conference on Software Engineering (ICSE)*, 2013, pp. 712–721.
- [10] O. Kononenko, O. Baysal, and M. W. Godfrey, "Code Review Quality: How Developers See It," in *Proc. 38th International Conference on Software Engineering (ICSE)*, 2016, pp. 1028–1038.
- [11] M. Bosu and J. C. Carver, "Impact of Peer Code Review on Peer Impression Formation," *Empirical Software Engineering*, vol. 19, no. 6, pp. 1745–1776, 2014.
- [12] N. Ayewah, D. Hovemeyer, J. Morgenthaler, J. Penix, and W. Pugh, "Using Static Analysis to Find Bugs," *IEEE Software*, vol. 25, no. 5, pp. 22–29, 2008.
- [13] Pylint Documentation, "Pylint — Python Static Code Analysis Tool," Available: <https://pylint.org>
- [14] S. Heckman and L. Williams, "On Establishing a Benchmark for Evaluating Static Analysis Alert Prioritization and Classification Techniques," in *Proc. IEEE International Symposium on Software Reliability Engineering*, 2011.